perform better than k-d trees in many situations, and especially for higher-dimensional data.

- Ss$^+$-trees improve the structure of these trees by using heuristics that reduce the nodes overlapping.
- You can trade the quality of search results for performance by using approximated similarity searches; there are several ways to do so, while guaranteeing that results will be within a certain approximation error from the true nearest neighbor/range searched.
- There are many domains where exact results for these searches are not important, because either we can accept a certain error margin or we don't have a strong similarity measure that can guarantee that the exact result will be the best choice.
- An example of such domain is similarity search in an image dataset.

---

[1] Convolutional Neural Network, a type of deep neural network that is particularly well suited to process images.

[2] A B-tree is a self-balancing tree optimized for efficiently storing large datasets on disk.

[3] A *memory page*, or just *page*,

[4] An attentive reader will remember that we already discussed the branching factor in chapter 2, for d-ary heaps.

[5] See, for instance, https://sqlity.net/en/2445/b-plus-tree/ .

[6] The centroid is defined as the center of mass of a set of points, whose coordinates are the weighted sum of the points' coordinates. If `N` is a leaf, its center is the centroid of the points belonging to `N`; if `N` is an internal node, then we consider the center of mass of the centroids of its children nodes.

[7] Since the height of the tree is at most $\log_m(n)$, if `m==M/2` (the choice with the largest height) $\log_{M/2}(n) \sim= \log_M(n)$.